

DEPARTAMENTO DE INFORMÁTICA

INF2102: PROJETO FINAL DE PROGRAMAÇÃO

Explorador de Espaço Vetorial para Imagens

Gabriel Ribeiro Gomes
ggomes@inf.puc-rio.br

Orientador:
Alberto Barbosa Raposo
abraposo@inf.puc-rio.br

Rio de Janeiro
Julho de 2026

1 O que é Recuperação de Imagens por Conteúdo (CBIR)

Recuperação de imagens por conteúdo (do inglês *Content-Based Image Retrieval*, CBIR) é a tarefa de buscar imagens semelhantes a uma imagem de consulta usando o próprio conteúdo visual, e não texto, etiquetas ou metadados associados [1]. A ideia central: converter cada imagem em um vetor numérico (um *embedding*) que captura suas características visuais, de modo que imagens parecidas fiquem próximas em um espaço vetorial e imagens diferentes fiquem distantes. A busca por semelhança vira, então, uma busca por vizinhos mais próximos nesse espaço.

Isso é importante por várias razões práticas e de pesquisa:

- **Busca sem rótulos:** encontrar imagens relevantes mesmo quando não há descrição textual, o que é comum em grandes acervos não anotados.
- **Rotulagem automática:** se uma imagem nova cai perto de um aglomerado de imagens de classe conhecida, pode-se propor um rótulo para ela. Esse é o princípio que motiva este trabalho e a dissertação associada.
- **Avaliação de representações:** a qualidade de um *embedding* pode ser julgada por quão bem ele agrupa imagens da mesma classe e separa classes diferentes.

O desafio central da área é obter representações que capturem semelhança *semântica*, e não apenas semelhança de pixels: duas fotos do mesmo objeto sob ângulos ou iluminações diferentes devem ficar próximas. Modelos modernos de visão como o CLIP [2] produzem embeddings genéricos fortes, e uma parte importante da pesquisa em CBIR é justamente medir o quão confiáveis esses embeddings são para uma tarefa concreta. Este programa serve exatamente a esse propósito: tornar visível e mensurável o comportamento de recuperação.

Estudo de caso e dados de exemplo. O sistema é genérico e funciona com qualquer imagem. O conjunto de amostra usado nas demonstrações vem de um projeto de monitoramento de embarcações na Baía de Guanabara (TecGraf PUC-Rio / Embraer), mas nada no sistema é específico desse domínio.

2 Breve Descrição

O **Explorador de Espaço Vetorial para Imagens** é uma ferramenta de recuperação de imagens por conteúdo. O programa indexa *embeddings* de imagem em um banco de dados vetorial e permite **explorar visualmente o espaço de representação**: projeta a galeria indexada em 2D/3D via PCA, aceita uma imagem de consulta, e mostra onde ela se posiciona em relação aos aglomerados (*clusters*) de cada classe. Além disso, prevê por votação *K-Nearest Neighbors* (KNN) [3] sobre os vizinhos recuperados a que classe a imagem consultada pertenceria, com um grau de confiança.

Principais funções

Função	Descrição
Indexação	Extraí <i>embeddings</i> de recortes de imagem com um modelo selecionável e os armazena no Milvus.
Projeção	Reduz os <i>embeddings</i> da galeria a 2D/3D com PCA para visualização.
Consulta visual	Projeta uma imagem nova no <i>mesmo</i> espaço da galeria e destaca seus vizinhos mais próximos.
Predição de classe	Vota, por KNN, a classe da imagem consultada e reporta a confiança.
Snapshot reproduzível	Exporta/reconstrói uma coleção a partir de um cache de <i>embeddings</i> (sem GPU).

Tabela 1: Funções oferecidas pelo programa.

Usuários visados: pesquisadores e estudantes de *CBIR* / visão computacional que precisam **conferir visualmente** se a recuperação e a classificação de suas representações estão coerentes, em vez de confiar apenas em métricas agregadas.

Natureza do programa: ferramenta utilitária funcional (prova de conceito madura), servindo de base experimental para uma dissertação de mestrado sobre rotulagem automática por recuperação.

Ressalva de uso: o programa **não** treina modelos e **não** é um classificador de produção. A predição KNN é uma *heurística de rotulagem por recuperação*, cuja confiabilidade é justamente o objeto de estudo. Os *embeddings* vêm de um modelo genérico externo (OpenCLIP), não de um modelo especializado no domínio.

3 Visão de Projeto

Esta seção apresenta quatro cenários (dois positivos e dois negativos) que orientam a intenção do criador e a interpretação do usuário.

3.1 Cenário Positivo 1: Conferir um agrupamento coerente

Marina, pesquisadora de CBIR, quer saber se um *embedding* genérico separa bem recortes de **Traineira** de **Rebocador**. Ela indexa a galeria de amostra, abre o explorador, escolhe a projeção 3D e vê quatro nuvens de cor razoavelmente distintas. Ela arrasta um recorte de **Rebocador** como consulta: o ponto vermelho cai **dentro** da nuvem de **Rebocador**, os 10 vizinhos exibidos são todos rebocadores, e o painel de predição mostra “**seria rotulada como Rebocador, 90% de confiança**”. Marina conclui, visualmente, que a representação é adequada para essa classe naquela faixa de tamanho.

Este cenário evoca as funções centrais: indexação, projeção, consulta e predição. Note que Marina não precisou de nenhuma métrica agregada: a resposta veio da posição do ponto e da concordância dos vizinhos.

3.2 Cenário Positivo 2: Trocar de modelo com garantia de consistência

Pedro desconfia que *patches* menores capturariam melhor embarcações pequenas. Ele reindexa a mesma galeria com `openclip-vit-b-16` em uma coleção separada. Ao abrir o explorador e selecionar essa coleção, a interface exibe “**modelo de embedding: openclip-vit-b-16**” e garante que qualquer consulta será *embeddada* com esse mesmo modelo. Pedro compara as duas projeções lado a lado e decide qual modelo separa melhor as classes, sem risco de comparar vetores de espaços diferentes.

Este cenário evoca a troca de modelos e a **garantia de consistência**: a coleção “*lembra*” com qual modelo foi construída, e o sistema recusa misturar espaços de *embedding*.

3.3 Cenário Negativo 1: Consulta com modelo incompatível

Ana tenta, via API, consultar uma coleção construída com `openclip-vit-b-32` forçando o modelo `openclip-vit-b-16`. O sistema **recusa** a operação com um erro 409 **Conflict** e a mensagem: “*a coleção foi construída com openclip-vit-b-32, mas a consulta pediu openclip-vit-b-16; os embeddings seriam incomparáveis*”.

Este cenário ilustra uma limitação **conhecida e desejada**: distâncias entre vetores de modelos diferentes não têm significado. O programa prefere falhar de forma clara a devolver um resultado silenciosamente incorreto. Não há como contornar isso pela interface, e é intencional.

3.4 Cenário Negativo 2: Recorte ambíguo mal classificado

João consulta com um recorte de **Traineira** pequeno e distante, capturado ao fundo de uma cena. O ponto de consulta cai na fronteira entre **Traineira** e **Navio de Carga Geral**, e a predição KNN retorna “**Navio de Carga Geral**” com alta confiança (um erro). Ao inspecionar os vizinhos exibidos, João percebe que todos são embarcações pequenas e distantes da mesma câmera: visualmente parecidas, apenas alguns *pixels*.

Este cenário expõe uma limitação diferente da anterior: para objetos muito pequenos, o *embedding* genérico captura mais o contexto de cena do que o objeto. O programa não esconde isso: ao contrário, a ferramenta **serve exatamente para tornar esse tipo de falha visível**, o que é um resultado de pesquisa, não um defeito de software.

4 Documentação Técnica do Projeto

4.1 Modelo de Arquitetura

O sistema é organizado em três camadas com dependência unidirecional (*frontend* → API → *backend*), mais os contratos de dados e a observabilidade compartilhados (Figura 1).

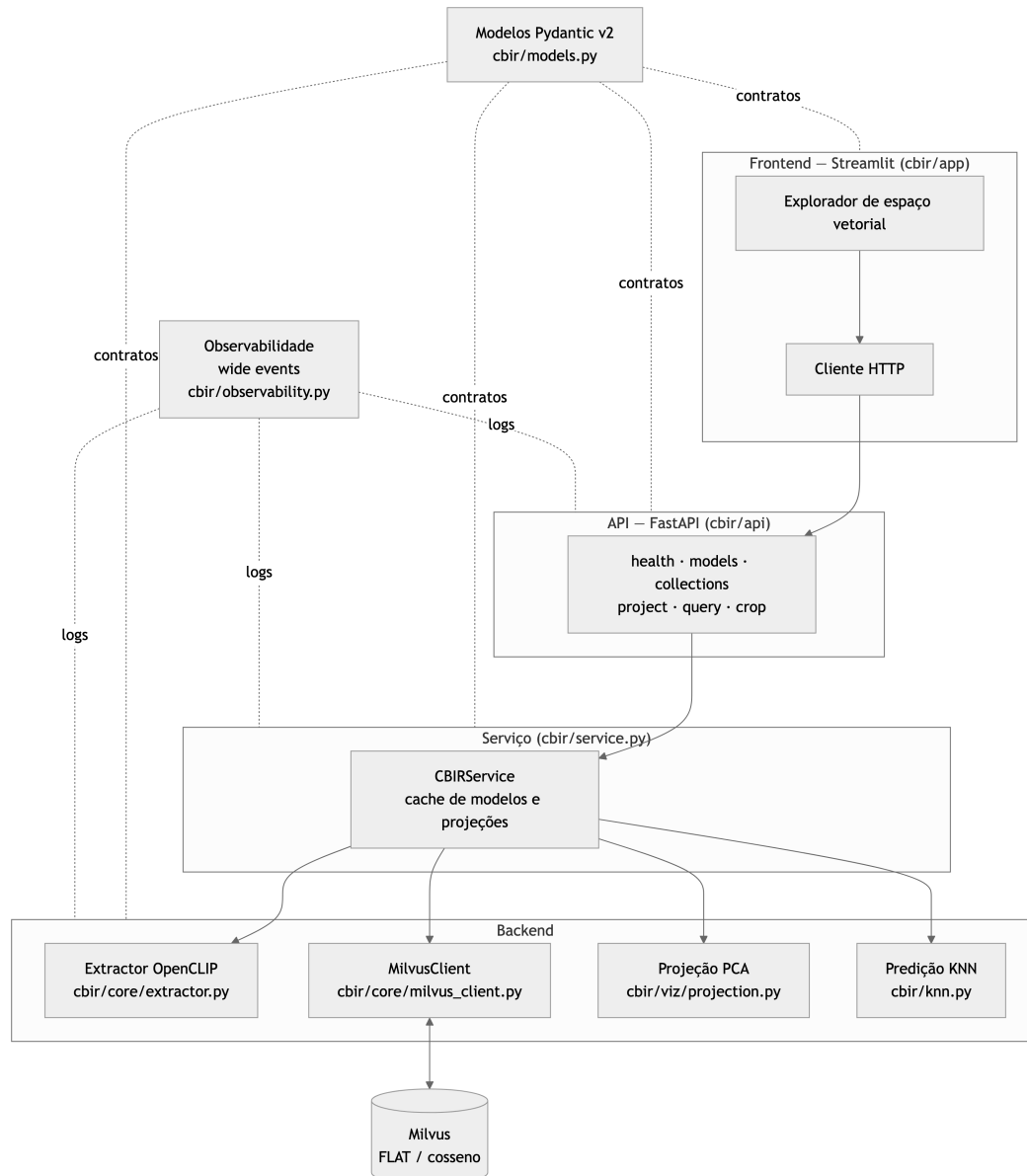


Figura 1: Arquitetura em três camadas. O *frontend* fala apenas com a API; a API delega ao serviço; o serviço orquestra o *backend* e o Milvus.

4.2 Modelo de Dados

A unidade canônica é o **recorte de bounding box**. Um *manifest* (JSONL, um registro por caixa) descreve cada item; o recorte é derivado em tempo de execução. Os *embeddings* e metadados são persistidos no Milvus. A Figura 2 apresenta as classes principais e a Figura 3 o diagrama entidade-relação das entidades persistidas.

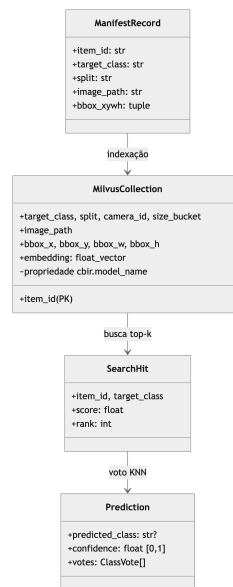
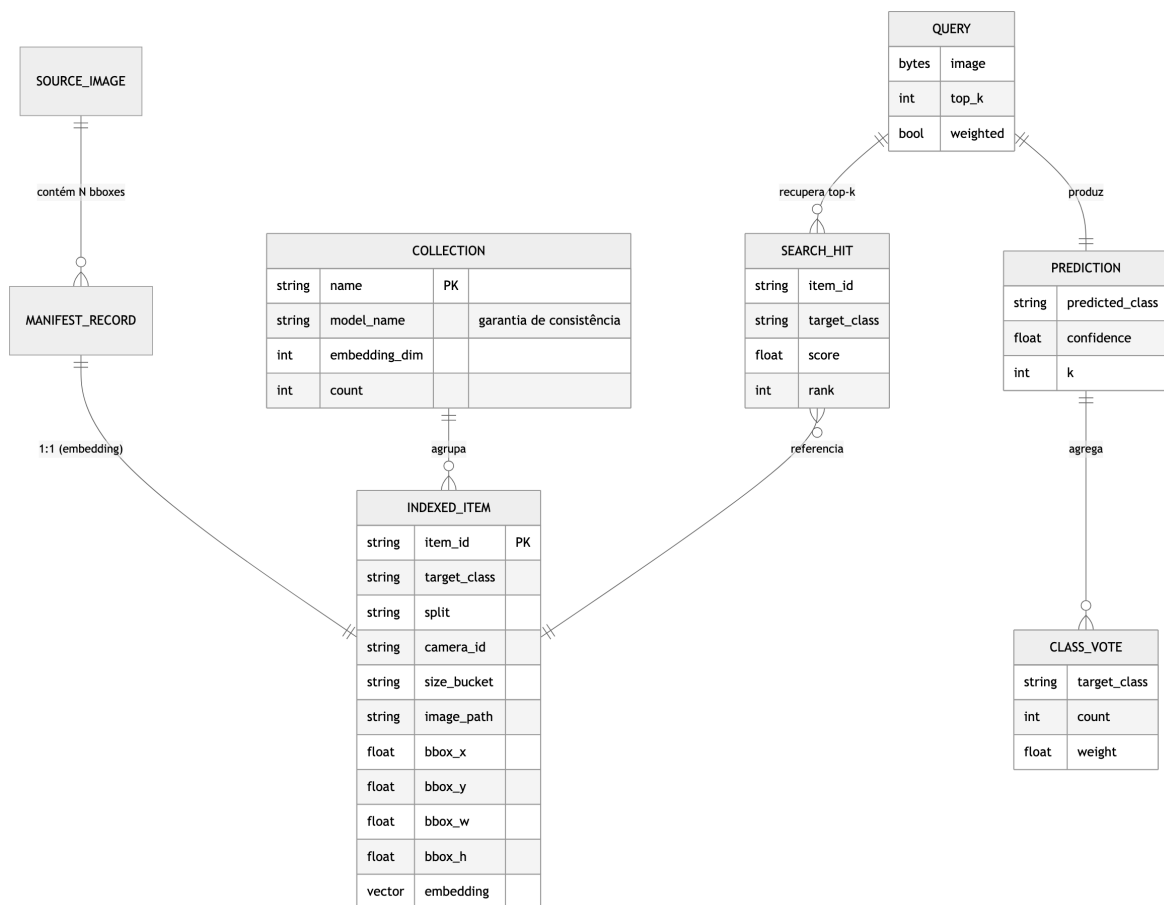


Figura 2: Principais classes de dados (contratos Pydantic v2).

Figura 3: Diagrama entidade-relação. Uma imagem-fonte contém N caixas; cada caixa vira um item indexado; cada consulta recupera k vizinhos, que alimentam uma predição.

O esquema de metadados carregado com cada vetor foi escolhido para ser exatamente o que o *frontend* precisa:

Campo	Uso
<code>target_class</code>	Cor do ponto no gráfico e voto KNN
<code>split</code> , <code>camera_id</code> , <code>size_bucket</code>	Facetas / <i>hover</i>
<code>image_path</code>	Servir o recorte como miniatura
<code>bbox_x..h</code>	Reconstruir o recorte quando necessário
<code>embedding</code>	Vetor para busca por cosseno
<i>propriedade</i> <code>cbir.model_name</code>	Garantia de consistência de modelo

Tabela 2: Esquema de metadados por vetor na coleção Milvus.

4.3 Fluxo de Indexação

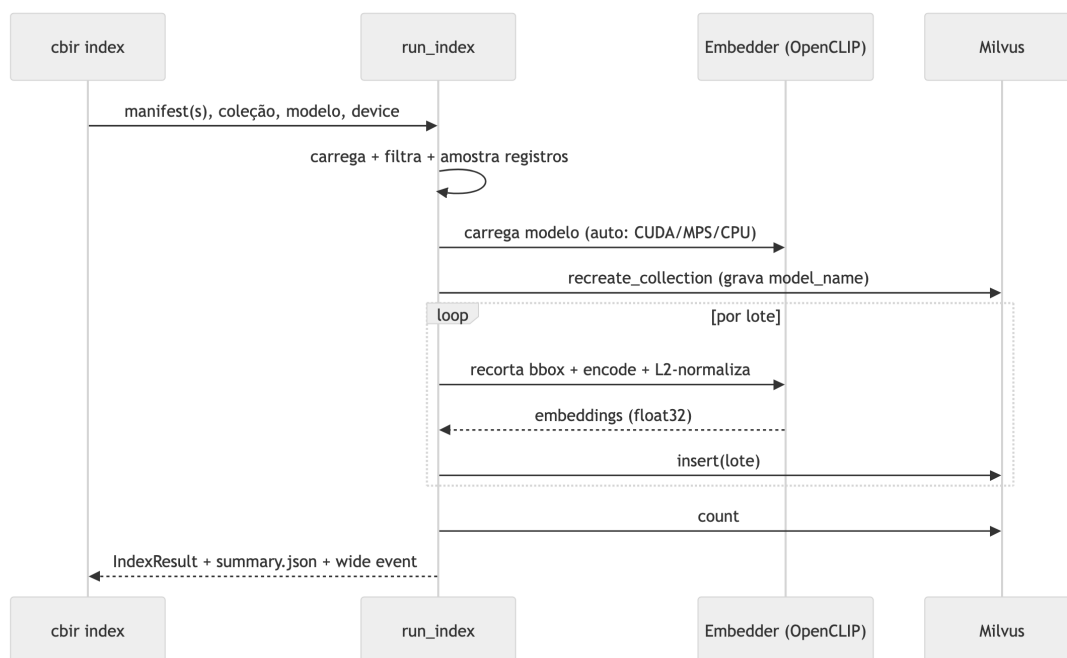


Figura 4: Fluxo de indexação: *manifest* → recorte → *embedding* → Milvus, com o modelo gravado na coleção.

4.4 Fluxo de Consulta

O fluxo de consulta (Figura 5) é o coração da ferramenta: embeda a imagem com o modelo da coleção, busca os vizinhos, projeta a consulta no mesmo PCA da galeria e vota a classe.

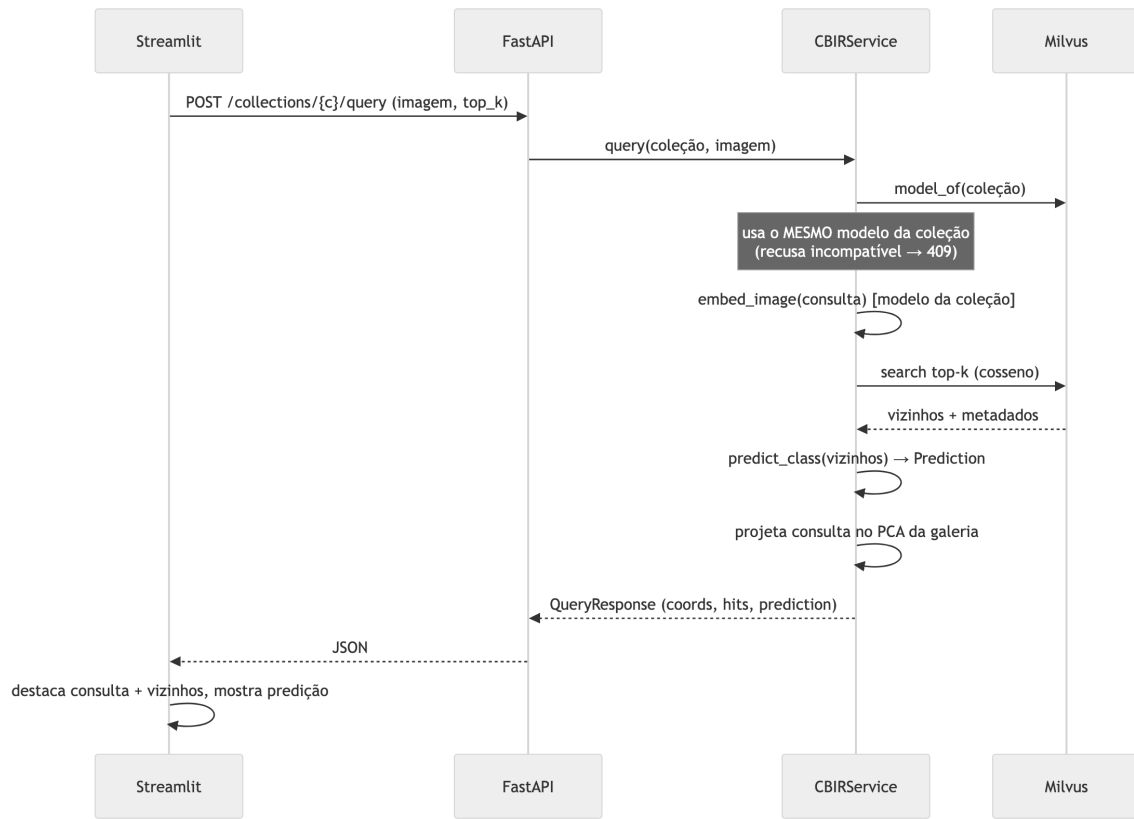


Figura 5: Fluxo de consulta. A consulta é sempre *embeddada* com o modelo da coleção; misturar espaços é recusado com 409.

4.5 Fundamentos: PCA e redução de dimensionalidade

Cada *embedding* produzido pelo modelo é um vetor de alta dimensão (no caso do OpenCLIP ViT-B/32, $d = 512$ dimensões). Esse espaço é o que permite medir semelhança com precisão, mas é **impossível de visualizar diretamente**: não há como desenhar um ponto em 512 eixos. Para inspecionar o espaço a olho nu precisamos projetá-lo em 2 ou 3 dimensões, aceitando que *parte da informação será perdida* nessa compressão. É um compromisso consciente: trocamos fidelidade por interpretabilidade visual.

A Análise de Componentes Principais (PCA) [4] faz essa redução buscando as direções de *maior variância* dos dados. Dada uma matriz $X \in \mathbb{R}^{n \times d}$ com n embeddings centrados (média zero), a PCA calcula a matriz de covariância

$$C = \frac{1}{n} X^T X \in \mathbb{R}^{d \times d},$$

e resolve seu problema de autovalores $Cv_i = \lambda_i v_i$. Os autovetores v_i (componentes principais) são ortogonais e ordenados pelos autovalores λ_i , que medem quanta variância cada direção captura. Para visualizar em k dimensões ($k = 2$ ou 3), tomamos os k autovetores de maior autovalor, formando $W_k = [v_1 \cdots v_k] \in \mathbb{R}^{d \times k}$, e projetamos

$$Z = X W_k \in \mathbb{R}^{n \times k}.$$

A fração de variância preservada é $\sum_{i=1}^k \lambda_i / \sum_{j=1}^d \lambda_j$; o relatório da interface mostra esse valor para deixar explícito o quanto foi perdido. Por ser uma transformação *linear* (a matriz W_k ajustada na galeria), a PCA projeta uma imagem de consulta nova no mesmo espaço com a mesma multiplicação $z = x W_k$, o que é a propriedade essencial para posicionar a consulta em relação aos aglomerados existentes.

4.6 Por que PCA (e não t-SNE/UMAP)

A projeção usa Análise de Componentes Principais (PCA) [4], uma redução linear de dimensionalidade. A alternativa seriam métodos não lineares como o t-SNE [5] ou o UMAP [6], mas eles não oferecem uma transformação exata para projetar um ponto novo (a consulta) no mesmo espaço da galeria já ajustada. A Tabela 3 resume a comparação.

Critério	PCA	t-SNE / UMAP
Projetar consulta nova no mesmo espaço	Exato e barato	Sem transform exato
Determinismo	Sim	Estocástico
Preserva distâncias globais	Sim (linear)	Foco em estrutura local
“Onde minha consulta cai vs. clusters”	Ideal	Enganoso

Tabela 3: Justificativa da escolha de PCA para a projeção.

A escolha de PCA é o que torna correta a pergunta central do programa: *onde esta imagem nova cai em relação aos clusters existentes?*, algo que exige aplicar exatamente a mesma transformação linear à consulta e à galeria.

4.7 A garantia de consistência de modelo

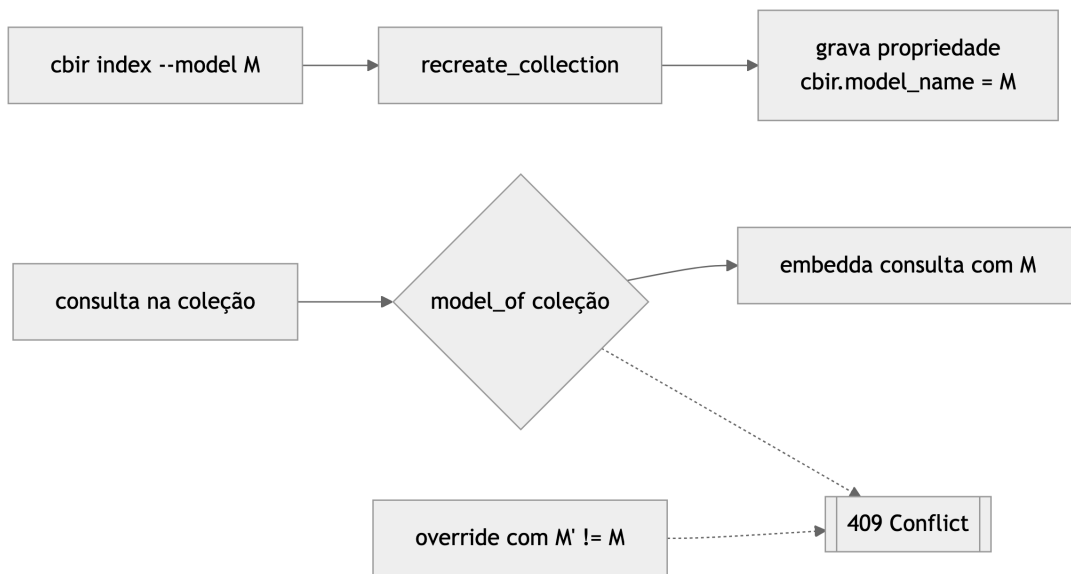


Figura 6: A coleção grava o modelo que a construiu; a consulta é sempre *embeddada* com ele, e qualquer *override* incompatível é recusado.

Um trecho do serviço que implementa a garantia:

```

collection_model = self.model_for_collection(collection)
chosen = model_name or collection_model
if collection_model and model_name and model_name != collection_model:
    raise ModelMismatchError(
        f"Collection {collection!r} was built with {collection_model!r}, "
        f"but the query requested {model_name!r}. "
        "Embeddings would be incomparable."
    )

```


4.8 Resolução de *device*

O sistema escolhe o melhor acelerador disponível e degrada com elegância (Figura 7), garantindo que o *mesmo* comando rode em qualquer máquina: CUDA, depois GPU Apple (MPS/Metal), depois CPU *multi-thread*.

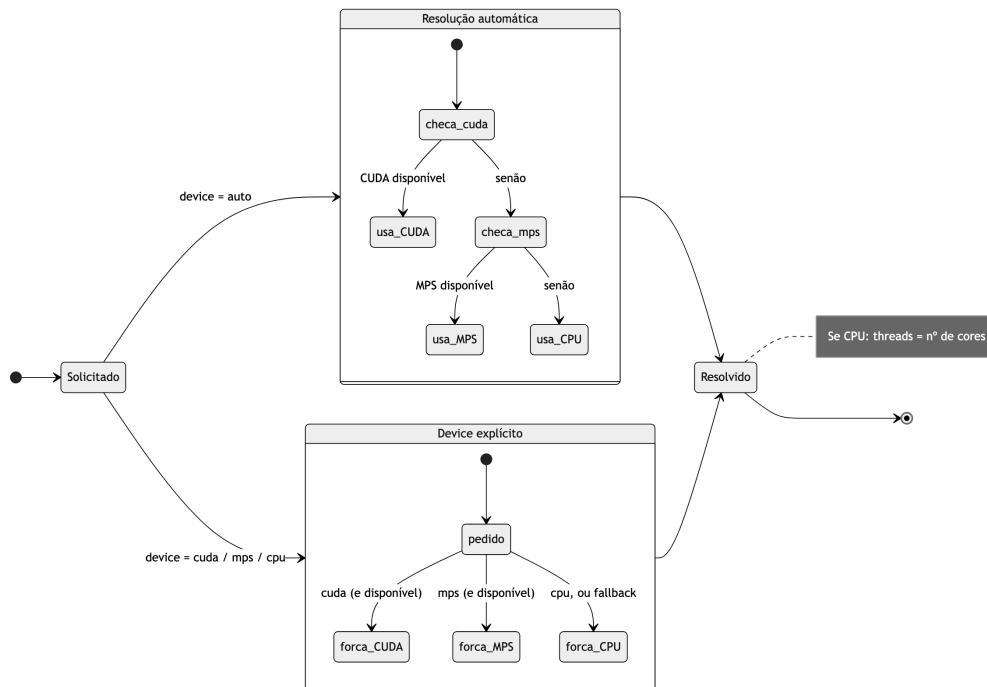


Figura 7: Máquina de estados da resolução de *device* (auto → CUDA → MPS → CPU).

4.9 Implantação (Docker Compose)

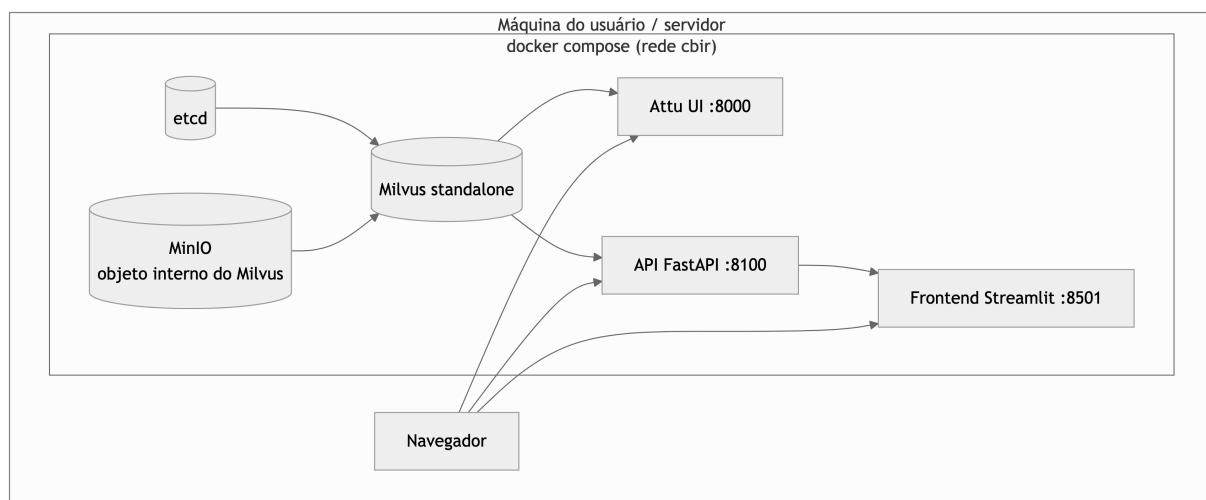


Figura 8: Diagrama de implantação. O perfil padrão sobe apenas o Milvus; o perfil *app* adiciona API e *frontend*.

4.10 Sobre o código

As principais tecnologias empregadas, com uma breve explicação de cada uma:

- **Python 3.13** com **uv**: linguagem do projeto e gerenciador de dependências/ambientes (rápido, com *lockfile* reproduzível).
- **OpenCLIP** [7]: implementação aberta do modelo CLIP, que converte uma imagem em um vetor (*embedding*) que captura seu conteúdo visual.
- **Milvus** [8]: banco de dados vetorial, especializado em armazenar *embeddings* e fazer busca por vizinhos mais próximos em larga escala.
- **scikit-learn (PCA)** [9]: biblioteca de *machine learning*; usamos o PCA para reduzir os vetores a 2D/3D preservando as direções de maior variância.
- **FastAPI**: *framework* web para APIs em Python, com validação e documentação automáticas; expõe o *backend* via HTTP.
- **Streamlit** com **Plotly**: Streamlit constrói a interface web em Python puro; Plotly desenha os gráficos de dispersão 2D/3D interativos.
- **Pydantic v2**: valida e tipa os dados que trafegam entre as camadas, garantindo que todas concordem sobre o formato.
- **ruff**, **mypy**, **pytest**: *linter*/formatador, verificador de tipos estático e *framework* de testes, respectivamente.
- **Docker Compose**: orquestra os serviços (Milvus, API, interface) em contêineres, para subir tudo com um comando.

Aspecto	Decisão
Linguagem	Python 3.13, gerenciada por uv
Contratos de dados	Pydantic v2 (validação nas fronteiras entre camadas)
Extração	OpenCLIP; recorte em <i>runtime</i> ; L2-normalização
Banco vetorial	Milvus <i>standalone</i> (índice FLAT, métrica cosseno)
Projeção	scikit-learn PCA (2D/3D), com degradação graciosa
API	FastAPI (endpoints finos sobre o serviço)
Frontend	Streamlit + Plotly (fala apenas com a API)
Device	auto: CUDA → Apple MPS → CPU <i>multi-thread</i>
Observabilidade	logging <i>stdlib</i> com <i>wide events</i>
Qualidade	ruff (lint), mypy (tipos), pytest (testes)

Tabela 4: Principais decisões de implementação.

Estratégia de comentários: *docstrings* explicam a *intenção* e o *porquê* de cada módulo/função; comentários em linha marcam decisões não óbvias (ex.: por que negar similaridade negativa no voto, por que PCA e não UMAP). Código óbvio não é comentado.

5 Testes

A suíte tem **30 testes** e foi desenhada para ser rápida e determinística: o *backend* é puro e testado sem Milvus nem *torch*, e a API é testada com um serviço falso (dublê), o que dispensa qualquer dependência externa e ainda permite verificar a garantia de consistência de modelo (resposta 409). Os testes rodam em poucos segundos.

Arquivo	O que cobre	Nº
test_knn	Voto KNN: conjunto vazio, unanimidade, maioria vs. voto ponderado, corte por k , similaridade negativa sem subtrair evidência, desempate determinístico	7
test_manifest	Contrato de dados: rejeição de item_id duplicado, campos obrigatórios, filtros por split/benchmark, amostragem determinística por classe, recorte de bbox com clip nas bordas	7
test_projection	PCA: dimensionalidade pedida, transform reproduz a galeria, vetor único, degradação graciosa, galeria vazia, determinismo	6
test_api	Contrato HTTP: health , coleções com seu modelo, projeção e 404, consulta feliz com predição, upload inválido rejeitado	6
test_models	Modelos Pydantic: limites de confiança, rank positivo, campos model_* , <i>round-trip</i> JSON	4

Tabela 5: Cobertura da suíte de testes (30 no total).

Trecho ilustrativo (o voto ponderado por similaridade pode inverter o vencedor em relação à maioria simples):

```
def test_weighted_vote_can_flip_the_winner() -> None:
    hits = [
        _hit("a", "Traineira", 0.30, 1),
        _hit("b", "Traineira", 0.31, 2),
        _hit("c", "Lancha", 0.99, 3),
    ]
    pred = predict_class(hits, weighted=True)
    assert pred.predicted_class == "Lancha"
    assert abs(pred.confidence - 0.99 / 1.60) < 1e-6
```

Execução completa (as três verificações devem ficar verdes):

```
uv run ruff check cbir/ tests/ # lint
uv run mypy cbir/ # tipos
uv run pytest # 30 testes
```

6 Manual de Utilização

O manual cobre os dois tipos de usuário visados: quem quer **rodar a demo** rapidamente e quem quer **indexar dados próprios**.

6.1 Instalação

```
uv sync # instala dependencias
docker compose up -d # sobe Milvus (etcd + minio + milvus + Attu)
# aguarde o Milvus ficar "healthy" (docker compose ps)
```

6.2 Tarefa A: Rodar a demonstração (sem GPU, sem baixar modelo)

```
uv run cbir seed --collection cbir_sample \
    --parquet cbir/sample_data/embeddings.parquet
uv run cbir api # terminal 1: API em :8100
uv run cbir app # terminal 2: frontend em :8501
# abra http://localhost:8501, escolha "cbir_sample" e envie um recorte
```

Exceções e problemas comuns:

- “*API not reachable*”: confirme que **cbir api** está rodando e a porta 8100 está livre.
- *Gráfico vazio*: rode **cbir seed** antes de abrir o app; a coleção precisa existir.

6.3 Tarefa B: Indexar dados próprios

```
uv run cbir index --manifest <caminho> --collection <nome> \
  --model openclip-vit-b-32 --device auto
# (opcional) snapshot reproduzível:
uv run cbir export --collection <nome> --model openclip-vit-b-32
```

Exceções e problemas comuns:

- “No records selected”: revise `--split` / `--benchmark-only` / `--per-class`.
- *Consulta recusada com 409*: a coleção foi construída com outro modelo; use o modelo da coleção.

6.4 Referência de comandos e endpoints

Comando	O que faz
<code>cbir sample</code>	Constrói o <i>dataset</i> de amostra comitável (recortes + <i>manifest</i>)
<code>cbir index</code>	Extraí <i>embeddings</i> de um <i>manifest</i> e indexa no Milvus
<code>cbir export</code>	Exporta os <i>embeddings</i> de uma coleção para um cache Parquet
<code>cbir seed</code>	Reconstrói uma coleção a partir de um cache Parquet (sem modelo)
<code>cbir api</code>	Sobe o serviço FastAPI
<code>cbir app</code>	Sobe o <i>frontend</i> Streamlit

Tabela 6: Comandos da CLI.

Método	Rota	Função
GET	<code>/health</code>	Verificação de vida
GET	<code>/models</code>	Modelos de <i>embedding</i> disponíveis
GET	<code>/collections</code>	Coleções indexadas + modelo + contagem
GET	<code>/collections/{n}/project</code>	Coordenadas PCA da galeria (2D/3D)
POST	<code>/collections/{n}/query</code>	Imagem → vizinhos + predição KNN + coords
GET	<code>/crop</code>	Serve um recorte por caminho de <i>manifest</i>

Tabela 7: Endpoints da API.

7 Verificação

O sistema foi validado de ponta a ponta na máquina de desenvolvimento (Apple Silicon, *device mps*).

Verificação	Resultado
Indexação da amostra (160 recortes, 4 classes)	160 itens em ≈ 29 s (MPS)
Projeção PCA 2D/3D da galeria	160 pontos; transform exato
Consulta ponta a ponta (upload → busca → KNN → projeção)	Predição coerente com confiança
Garantia de consistência de modelo	Recusa (409) confirmada em teste
<code>ruff</code> / <code>mypy</code> / <code>pytest</code>	Limpo / limpo / 30 testes passando
Reconstrução via cache (<i>seed</i>)	Coleção recriada em ≈ 3 s sem GPU

Tabela 8: Resultados da verificação de ponta a ponta.

Nota sobre escala da visualização. A verificação usou 160 vetores. O gargalo prático da ferramenta não é o Milvus nem o PCA, e sim a renderização no navegador: o Plotly desenha os pontos como SVG por padrão, o

que mantém a interação (rotação 3D, *hover*, *zoom*) fluida até a ordem de alguns milhares de pontos e começa a degradar acima disso. Escalar para dezenas de milhares exigiria trocar para renderização WebGL (*Scattergl*, que suporta ~100 mil pontos) ou amostrar/agregar a galeria antes de plotar. Esses são limites esperados da abordagem de renderização, não medições deste projeto, e ficam como trabalho futuro natural.

8 Conclusão e Trabalhos Futuros

O programa entrega um sistema CBIR funcional e reproduzível, com três camadas bem separadas, uma garantia explícita de consistência de modelo, e uma interface visual que torna concreta a pergunta central da pesquisa: *uma imagem de classe A se posiciona perto de outras da classe A e longe das demais?* Como trabalhos futuros, destacam-se: (i) a comparação sistemática entre modelos de *embedding*; (ii) o uso de *embeddings* derivados de um detector treinado no domínio; e (iii) a avaliação quantitativa da confiabilidade dos limiares de similaridade para rotulagem automática, que é o objeto da dissertação.

Referências

- [1] Arnold W. M. Smeulders et al. “Content-Based Image Retrieval at the End of the Early Years”. Em: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.12 (2000), pp. 1349–1380. DOI: [10.1109/34.895972](https://doi.org/10.1109/34.895972).
- [2] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. Em: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 2021, pp. 8748–8763.
- [3] Thomas M. Cover e Peter E. Hart. “Nearest Neighbor Pattern Classification”. Em: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. DOI: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).
- [4] Ian T. Jolliffe. *Principal Component Analysis*. 2ª ed. New York: Springer, 2002. DOI: [10.1007/b98835](https://doi.org/10.1007/b98835).
- [5] Laurens van der Maaten e Geoffrey Hinton. “Visualizing Data using t-SNE”. Em: *Journal of Machine Learning Research* 9 (2008), pp. 2579–2605.
- [6] Leland McInnes, John Healy e James Melville. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction”. Em: *arXiv preprint arXiv:1802.03426* (2018).
- [7] Gabriel Ilharco et al. *OpenCLIP*. Disponível em: https://github.com/mlfoundations/open_clip. 2021. DOI: [10.5281/zenodo.5143773](https://doi.org/10.5281/zenodo.5143773).
- [8] Jianguo Wang et al. “Milvus: A Purpose-Built Vector Data Management System”. Em: *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2021, pp. 2614–2627. DOI: [10.1145/3448016.3457550](https://doi.org/10.1145/3448016.3457550).
- [9] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. Em: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.